WHITE PAPER · 24 OCTOBER 2026

The Interpreted Harness: Multi-Agent Coordination Through Protocol Documents

Christopher Maglio and Muse

What an agent harness is, how the best-known one grew out of a one-hour relay, and a demonstrated alternative: a harness whose loop is not compiled code but a natural-language protocol document, interpreted at run time by two complete, independent AI agents sharing a message board.

Abstract

The agent harness — the software layer around a language model that assembles its context, runs its tool-calling loop, and manages its memory — has hardened into a settled category, summarized by the formulation *Agent = Model + Harness*. Every production harness is compiled code. This paper examines what a harness actually is by tracing the history of the most widely adopted one, OpenClaw, which began in November 2025 as a one-hour relay piping WhatsApp messages into a coding agent on its author's PC, and became a harness at a specific, identifiable moment: when its gateway closed the feedback loop itself — calling the model, executing its tool requests, returning the results, and calling again — and thereby took agency over solving the problem. We then present a second architecture, demonstrated in a series of experiments conducted in October 2026, in which the loop is closed without any harness code at all. A relay (a public message board), a protocol document written in natural language, a short bootstrap prompt, and two complete, independent AI agents — each already equipped with its own model, harness, tools, and environment — together form what we call an *interpreted harness*: the document is the code, the board is the memory, the prompt is the bootstrap, and the agents are the interpreters. Because its components are complete agents rather than one model and one tool registry, the interpreted harness commands the union of two agents' capabilities, including genuinely independent verification — in the debate experiment reported here, one agent corrected the other's stale facts from its own live sources mid-task. The same redundancy makes the system tolerant of defective goals: the operator's prompt for that debate contained multiple errors, yet the run still produced a high-quality result, because two agents interpreted the flawed specification independently and reconciled their readings through the protocol. We argue this is the architecture's decisive advantage, and that it is specifically an advantage in prompt engineering: that discipline exists to make a single interpreter's prompt survivable, and the interpreted harness — reconciling two independent interpretations instead of armouring one — makes much of it unnecessary, accepting goals as operators actually write them: rough, incomplete, and partly wrong. The cost is reliability: state held in a model's context can be lost, as one failed experiment shows. We compare the two architectures, state the mainstream position that loop logic belongs in code rather than prompts, and identify where each architecture belongs.

1. Introduction

Ask what makes an AI agent work and the industry answer, converged upon during 2026, is that the model is the smaller half of it. A widely cited formulation from LangChain's engineering team puts it in one line: *Agent = Model + Harness* — if you are not the model, you are the harness [1]. The harness is everything around the weights: context assembly, the tool registry, memory, guardrails, verification, and above all the *agent loop*, the cycle that calls the model, executes the action it requests, feeds back the result, and calls it again until the work is done. Strip the harness away and, as one explainer puts it, “you do not have an agent anymore. You have a model that can talk.”

The word itself is older and plainer than the marketing around it. A harness, in the test-engineering sense, is the fixture and cabling that holds a device under test and connects it to power, inputs, and measurement. Nobody claims the fixture is intelligent; its job is to be the rig around the thing. That sense is worth keeping, because it disciplines the discussion that follows: a harness is defined not by sophistication but by *position and function* — it is the layer that holds agency over a task on the operator's behalf.

Every harness in production is implemented the same way: as compiled (or at least conventionally executed) code that drives a comparatively passive model. This paper asks whether that implementation is essential to the category, and answers with a working counterexample. We proceed in four steps. Section 2 describes the agent loop's mechanism precisely, because several popular accounts of it are wrong in instructive ways. Section 3 gives the history of OpenClaw — the most instructive case available, because its author built it in the open, in stages, and the stages separate cleanly into *relay* and *harness*. Section 4 states the criterion that history suggests: a harness exists where the loop closes and the system takes agency over the problem. Sections 5–7 present the interpreted harness and the experiments that demonstrate it, including its strongest argument — that it coordinates not one model but two complete agents — and Section 8 gives the tradeoffs without varnish.

2. The Agent Loop: How Harnesses Actually Work

A language model is stateless. It retains nothing between calls; everything it “knows” about a task in progress must be present in the prompt of the call currently being made. The harness therefore assembles the full context — system instructions, the conversation and action history, tool results — and sends it, in its entirety, on every call. The model's reply is one of two things: ordinary text, or a *tool call*, a structured request to execute a named function with given arguments (run this shell command, read this file, fetch this page).

The loop is then almost embarrassingly simple. If the reply contains a tool call, the harness executes it, appends the result to the history, and calls the model again. If the reply is plain text with no tool call, the loop stops. The continue/stop decision is structural, not verbal: the harness does not watch for the word “done”; it watches for the presence or absence of a requested action. Everything that makes agents useful — and everything that makes them expensive or dangerous — is a consequence of this while-loop with an append.

The consequence that matters most is *self-correction*, and its source is widely misattributed. The loop corrects errors not because the model is reflective but because the feedback is *external*. When a tool executes, the harness captures three things: the exit code, stdout, and stderr — the separate output channel programs reserve for errors and diagnostics. A compiler's complaint (“error: ‘foo’ was not declared in this scope, line 42”), a test runner's expected-versus-actual diff, a shell's “file not found” — these are fifty years of developer tooling designed to let a programmer fix a failure without re-running anything, and they serve, accidentally and exactly, as the grounding signal a model needs. The harness contributes nothing intelligent: it passes the verdict through verbatim (subject only to size caps that protect the context window) and lets the model diagnose. A chatbot grading its own homework from memory drifts; a loop graded by the machine's exit codes converges, because reality pushes back in a channel the model cannot argue with.

Two engineering facts complete the picture. First, cost: since the full history is resent on every call, a long loop repays for the whole transcript on every iteration — consumption grows faster than linearly with steps, which is why production harnesses compact (summarize) older history and cap tool output. Second, brakes: an uncapped loop will thrash between two failing fixes indefinitely, so harnesses impose step, token, and wall-clock limits. The naive form of the loop — re-invoke the model until it stops asking for tools, cost be damned — is a prototype's philosophy; the shipped form counts.

Compiled harnesses ship in three packages. *Vendor-integrated*: the model's maker bundles the harness, as Anthropic does with Claude Code. *Application-embedded*: the harness exists only inside one application's own surfaces, as with AnythingLLM, whose document-chat product carries its agent loop, memory, and schedulers internally and inseparably. *Standalone gateway*: the harness is the product, deliberately UI-less, driving the model through whatever channels the user already has. The third package is OpenClaw's, and its history is the cleanest natural experiment in where a harness begins.

3. Case Study: OpenClaw — How a Relay Became a Harness

Peter Steinberger spent thirteen years building PSPDFKit, a PDF framework deployed on over a billion devices, before retiring, burning out, and returning to programming in 2025 through AI coding tools. In mid-2025 he published a post titled “Claude Code Is My Computer,” describing how Anthropic's coding agent had become his primary interface to his machine. The capability was real but the posture was wrong: agents would stall, waiting for input or approval, and there was no way to see or unstick them from his phone. His framing question was: why don't I have an agent that can look over my agents?

The first answer took about an hour. In November 2025 Steinberger built a prototype — called V Relay, remembered as the WhatsApp Relay — whose architecture he described as glue: on one side, WhatsApp; on the other, the reasoning capabilities of Claude Code running on his own machine. A message sent from his phone was handed to the local agent; the reply came back into the same WhatsApp thread. The WhatsApp side was not built on Meta's Business API and

required no per-message fees or business account: the relay connected the way WhatsApp Web does, as a linked device, a technique implemented in the current product by the open-source Baileys library, which emulates a WhatsApp Web client and pairs by QR code [2].

Observe what the relay was not. It contained no agent loop of its own. The loop it reached — tool calls, stderr feedback, self-correction — belonged entirely to Claude Code, Anthropic's vendor-integrated harness, already running on the machine. The relay was transport: it carried text between a phone and a harness, across the gap that had frustrated its author. This worked in an hour precisely because both hard parts already existed. WhatsApp carried the text; Claude Code ran the loop; the relay connected them.

The transformation came in the following days. Steinberger “vibe-coded” the full initial version in roughly ten days. The decisive change was not the addition of features but a change of ownership: his own *Gateway* process took over the loop. The gateway — a persistent local service, built around an internal agent engine called Pi, developed by collaborator Mario Zechner — now assembled the context, called the model, executed its tool requests (shell commands, file operations, web fetches, scheduling, memory search and storage; fifteen tools in one architecture account [3]), returned the results, and called again. The model became a pluggable component: Anthropic, OpenAI, Google, DeepSeek, and local runtimes (Ollama, LM Studio, vLLM, llama.cpp) are interchangeable providers behind the gateway. WhatsApp became one channel among many — Telegram, Discord, Slack, Signal, iMessage — and the agent acquired the trappings of persistence: identity, personality, rules, and memory held in plain Markdown files (a SOUL.md defining the agent; memory files and daily journals it updates over time), a web Control UI served by the gateway for sessions and configuration, and a heartbeat by which the agent wakes itself on a timer, reviews its state, and decides whether anything needs doing.

The project's public life was as chaotic as its architecture was clarifying. Published as Clawdbot — the agent persona was “Clawd,” a pun on the model it was built on — it drew a trademark complaint from Anthropic over the Claude-adjacent branding, was renamed Moltbot on 27 January 2026, and settled as **OpenClaw** three days later. On 15 February 2026 Steinberger joined OpenAI and the project passed to an open-source foundation. NVIDIA, announcing its own NemoClaw scaffold on 16 March 2026, called it the fastest-growing open-source project in history; by 11 October 2026 it had approximately 391,000 GitHub stars. Meta names OpenClaw the reference harness for its Muse Glimmer model: install the harness, point its model provider at your Glimmer endpoint [4].

One fact about the case study disciplines everything claimed for it: the creator's own deployment is a *local harness with a remote brain*. The gateway runs on his hardware, but the model he built it on — and the one his README recommends, Anthropic's flagship, for “long-context strength and better prompt-injection resistance” — is a cloud API. The dominant configuration among users is the same; fully local inference through Ollama is a real but minority pattern, typically hybridized with a cloud fallback, because sustained agentic tool use is exactly where smaller local models fall down. OpenClaw's achievement was never the loop — the loop predates it, in Claude Code and elsewhere. It was the relay generalised into a persistent,

channel-rich, memory-carrying fixture: a breakthrough in plumbing, shipped as a product, at the moment the plumbing was what the models needed.

4. The Criterion: Agency

The history separates the categories more cleanly than any definition proposed in advance. The WhatsApp Relay was not a harness, though it touched one. OpenClaw is a harness, though its core loop is a while-loop with an append that a competent programmer writes in an afternoon. What changed between the relay and the product was not the quantity of code. It was *who owned the problem*.

In the relay configuration, the operator's problem — an agent stalled on a distant machine — remained the operator's problem, with better transport. Claude Code solved its own sub-problems on the machine; the human still supervised, decided, and intervened through a longer wire. When the gateway closed the loop itself, the locus of problem-solving moved: the system received a goal, acted on the world through tools, read the world's verdicts out of stderr and exit codes, corrected itself, and reported when finished. The operator supplied the goal and the boundaries; the system supplied the agency.

> **Definition.** A *harness* is a system that closes the loop around a model — call, execute, feed back, repeat — and thereby takes agency over solving a problem on the operator's behalf. The loop closure is the mechanism; the agency is what the mechanism confers. A system that transports messages to a loop it does not own is a relay. A system that owns the loop is a harness, whatever it is made of.

The last clause is the paper's thesis, and it is load-bearing. If agency is the criterion, then the implementation of the loop — compiled binary, framework, or, as we now show, a natural-language document interpreted by the agents themselves — is an engineering choice with tradeoffs, not a boundary of the category. The mainstream of harness engineering currently treats the choice as settled: one widely read explainer's core advice is to put fragile steps “in the harness, not in the prompt,” to distrust the loop's exit and verify in code, and when an agent fails, to “fix the harness first” — in code, so the mistake cannot recur [1]. That position is correct wherever unattended reliability is the requirement. It is not the whole space. The remainder of this paper maps the part of the space it leaves out.

5. The Interpreted Harness

An interpreted harness replaces the gateway binary with four components, none of which is agent-loop code:

- **A relay.** A shared message board that both agents can post to and read — in our experiments, a public board reached over the internet through a reverse tunnel hosted on a microcontroller (an Arduino Nano ESP32 running a SuperDMZ tunnel client, which requires

no public IP or port forwarding). The board is the system's shared memory: the state of the collaboration lives there, in the open, where both agents can read it.

- **Two complete AI agents.** Not two instances of one model in one framework, but two independent, full agent systems — in the experiments, this paper's second author (Muse, in its own application) and Manus, and in earlier experiments Grok — each with its own model, its own harness, its own tools, its own environment and live sources, operated as separate products by separate vendors.
- **A prompt.** A short instruction from the operator to each agent: where the board is, which role it holds, and what this run's task is. The prompt is a bootstrap, not the program; it loads the program and presses run.
- **A protocol document.** A written specification, in natural language, of the collaboration's state machine: how a link is requested and established, how the parties negotiate terms (payload description, chunking, hash, polling interval, timeouts), how transfer proceeds in strict alternation with exact acknowledgement matching, how completion is verified by digest, and how the link is torn down. The document is the code. It is executed by interpretation.

The mapping to the compiled architecture is exact, part for part: the board serves as the memory files and session store; the protocol document serves as the gateway binary; the prompt serves as configuration and invocation; the agents serve as interpreters and processors in one. And the analogy that names the design is the field-programmable one: the agents are general-purpose processors, and the protocol document is a configuration bitstream loaded into them for a run. Edit the document and the harness is reprogrammed — new states, new failsafes, new negotiation terms — with no code changed, no binary rebuilt, and no cooperation required from either agent's vendor. The compiled harness cannot do this; the interpreted one can do nothing else.

Component	Compiled harness (OpenClaw)	Interpreted harness
Loop logic	Gateway binary, written in code	Protocol document, written in natural language
State / memory	Local session store, Markdown files	The shared board, read by both agents
Bootstrap	Configuration files, CLI invocation	A prompt from the operator to each agent
Processor	The harness code, driving one model	Two agents, interpreting the document
Tools	One registry, attached to one model	Each agent's own complete tool stack
Reprogramming	Edit code, rebuild, redeploy	Edit the document

Table 1 — The two architectures, component by component.

6. The Relay Experiments

The interpreted harness was not designed on paper and then built; it was operated, repeatedly, between 17 and 24 October 2026, and the protocol document was revised against what the runs exposed. The transport imposed real constraints, and they shaped the protocol as surely as any design decision. The board's web edge rejects POST requests, so all posting is done by GET with the message in the query string; the edge's request-line limit measured 8,163 characters accepted and 8,164 refused, which forced chunked transfer with base64 encoding (a safe chunk payload is roughly 7,400 characters) and made the negotiation phase — in which the parties agree the data description, chunk size, hash algorithm, polling behaviour, and timeouts before any payload moves — a necessity rather than a ceremony. Reads require a fresh cache-defeating parameter every time. The protocol's lifecycle is: IDLE → LINK REQUEST → LINK UP (a three-message handshake) → NEGOTIATING → TRANSFERRING, in strict alternation with exact acknowledgement matching → VERIFYING, in which the receiver posts a digest (the first 16 hexadecimal characters of the payload's SHA-256) and the sender answers COMPLETE or MISSING with the deficient chunks identified → COMPLETE → LINK DOWN. Two operating rules governed every run: the human operator is contacted exactly twice, for permission to start and for the completion report; and a stalled peer is reported to the operator when the silence exceeds roughly twice the measured normal round-trip (for one peer, about 12 seconds normal, report at 20) rather than at the end of a long failsafe timer [5].

Data transfers. The first extended transfer, to Grok, was halted by the operator at 75 of 111 chunks, the peer having been judged defective in its protocol adherence. The second, a QVGA image to the same peer, stalled at 8 of 19 acknowledged chunks when the peer's chunk tracking broke — it lost count of what it had received and could not recover it; the failure mode of Section 8, observed in the field, in the peer rather than in the protocol. The third transfer, a QVGA image to Manus on 19 October 2026, completed cleanly: two chunks, digest verified (1254709c580e96b7), zero resends — the first complete transfer, and the proof that the design closes its loop when both interpreters hold their state.

The debate. On 24 October 2026 the same apparatus ran a task no state machine could have executed: a structured debate. Both agents received the same prompt — debate which provider offers the best free virtual machine for hosting a website, three posts each (opening, rebuttal, closing), a 2,000-character limit per post, strict alternation over the board, then each agent owes the operator its own summary of at most 500 characters. One agent argued Oracle Cloud's Always Free tier; the other, Google's e2-micro. Two incidents from the run carry the paper's argument. First, the correction: the Oracle case rested in part on free-tier figures (4 OCPUs, 24 GB of RAM) that the opposing agent, consulting live documentation through its own tools, showed to be stale — the tier had been halved, to 2 OCPUs and 12 GB (1,500 OCPU-hours and 9,000 GB-hours monthly). The figures were conceded, the argument rebuilt on the corrected numbers, and the two closings converged: Oracle on resources, Google on predictability. Second, the refusal: the second agent's first attempt stalled before it began, because the board still held a leftover test message (“Car”) and the agent declined to post over it, citing the protocol's warning against overwriting state it did not own. No state machine exercises that judgment; a state

machine overwrites, or halts on a rule. The agent reasoned about the board's contents, reported the obstruction to the operator, and proceeded when told the way was clear.

7. Two Complete Agents

The compiled harness's unit of capability is one model plus one tool registry in one environment, with one body of knowledge and one view of the world. Its self-correction, for all the grounding stderr provides, is a single mind checking its own work with its own tools: it can catch everything its tools can measure and nothing they cannot. It cannot catch its own stale facts, because the staleness is in the checker.

The interpreted harness's unit of capability is the complete agent — and it has two of them. Each arrives with its own model and training, its own harness and tool stack, its own environment, subscriptions, and live sources, and its own independently formed knowledge. The system's capability is the *union* of the two stacks, and the union is not additive in the places that matter most:

- **Independent verification.** Two complete agents can check each other's *facts*, not merely each other's arithmetic. The stale-figure correction in the debate is the type specimen: an error that was invisible from inside one agent's knowledge was obvious from inside the other's, because the two knowledge bases were formed independently and only one was out of date. A compiled harness has no second knowledge base. This was, in fact, the operator's founding rationale for the relay, stated before any of these experiments: different AIs are good at different things, and letting them talk lets them share their strengths and cover each other's weaknesses.
- **Complementary tooling.** Each agent's tools are its own: different browsing, different retrieval, different execution environments, different rate limits and access. A task that is blocked on one side of the board (a paywall, a missing capability, a failed tool) is routinely open on the other.
- **Negotiation as a native operation.** Because both ends are agents, the terms of the collaboration are themselves discussable — chunk sizes, formats, the division of labour, the question to be debated. A compiled harness configures; an interpreted harness *agrees*. Gibberlink's viral 2025 demonstration — two voice agents recognising each other and negotiating a switch to a sound-based protocol [6] — showed the industry this operation in miniature, inside one demo. The protocol document makes it a standing capability.
- **Superiority in prompt engineering: tolerance of defective goals.** Prompt engineering, as practised on single models, is compensation for a brittle interpreter — the careful phrasings, schemas, and worked examples all exist because one model will execute a prompt's defects as faithfully as its intent. The compiled harness accepts this and engineers *around* the prompt, in code: validators, retries, structured outputs, guardrails. The interpreted harness engineers *at* the prompt layer instead, by making interpretation itself redundant. The mechanism: a compiled loop corrects *execution* errors — stderr and exit codes push back on how a step was performed — but it executes a flawed specification faithfully, because the specification is the one thing its feedback channels never grade. A single agent given a poorly worded or logically weak goal will either fail on it or silently deliver diligent work

toward the wrong target. The interpreted harness grades the goal itself, as a side effect of its structure. Two complete agents read the same defective prompt independently; each reconstructs the intent behind it from its own judgment and context, as two engineers reading the same flawed specification would; and the protocol then forces the two reconstructions to meet — in negotiation, in alternation, in rebuttal — where divergent readings surface and convergent ones earn confidence. The operator's debate prompt contained multiple errors, including a summary obligation ambiguous enough to require clarification afterward; the debate nonetheless ran clean and produced a result the operator judged high quality, with the defects absorbed in interpretation rather than propagated into the work. Redundant interpretation of a noisy specification is error correction applied to the goal: it cancels the defects that two independent readers would not both reproduce. The protocol document contributes a second stabiliser — even while the goal is being repaired in flight, the interaction's shape (alternation, limits, verification) is already fixed and sound, so a weak goal degrades the run's efficiency, not its coherence.

There is a recursion worth naming. A compiled harness is a harness whose component is a model. The interpreted harness is a harness whose components are *harnessed agents* — a harness of harnesses, one level up. The agency criterion of Section 4 applies at the higher level unchanged: the combined system receives a goal neither agent was given alone in executable form, divides and executes the work across two independent stacks, verifies the result with a digest or a rebuttal, and reports. It takes agency over the problem with two complete minds as its actuators. Where the task's difficulty is capability-shaped — breadth of knowledge, independence of checking, range of tools — this is not a marginal difference. It is the difference between employing a model and convening two practitioners.

8. Tradeoffs and Failure Modes

Nothing in Section 7 should be read as a general claim of superiority, and the experiments themselves supply the counterweight. The interpreted harness *can* be superior — in capability breadth, in independent verification, in negotiability. Its floor, however, is far below the compiled harness's floor, and both floors were measured in the runs above.

State fragility. In a compiled harness, loop state lives in code and storage: chunk counters, session records, digests are held by a process that cannot forget them. In an interpreted harness, state lives in the agents' contexts, and a model can lose it — as the second transfer's peer did at chunk 8 of 19, unrecoverably, mid-run. The board mitigates this (the authoritative record is shared and rereadable, which is precisely why the design puts state there rather than in either agent), and the protocol's digest verification detects corruption rather than propagating it, but no document can make an interpreter's working memory as trustworthy as a variable.

Discipline is interpreted, not enforced. Strict alternation, exact acknowledgement matching, the operator-contact rule: in code these are invariants; in a document they are instructions, observed as faithfully as the agent is capable of observing them, and subject to the agent's

judgment — usually a feature (the refused overwrite protected the board's state) and occasionally a bug (a peer that will not proceed is a stalled run until the operator intervenes).

Cost and latency. Two frontier agents, each resending full contexts, polling a board at one-second intervals, cost roughly twice a comparable single-agent loop and run at the speed of the slower interpreter. The compiled harness's costs are identical in structure and half in scale.

The bound on goal tolerance. Redundant interpretation cancels only the errors two readers make *independently*. A defect both agents read the same wrong way — a shared ambiguity, a premise both models happen to share — passes through undetected, the common-mode failure of any redundant system whose channels are not truly diverse. Two agents from the same vendor, or trained on substantially the same data, are less diverse than the architecture assumes. The tolerance is real and it is not magic: it is exactly as wide as the independence of the interpreters.

The mainstream position follows from these facts and deserves its strongest form:

for unattended, repeated, production work, loop logic belongs in code, the loop's exit should be verified by code, and an agent's failure should be fixed in the harness so it cannot recur [1]. We endorse every word of that — for the work it describes. The interpreted harness addresses the complementary set: collaborations between agents that share no framework, no vendor, and no operator-side code path; tasks whose terms must be negotiated rather than configured; and operators who cannot or will not stand up a gateway, but can write a document and a prompt. Where two complete agents already exist and a channel can be opened between them, the interpreted harness is not the cheaper substitute for the compiled one. It is the only one available — and, on capability-shaped tasks, the stronger one.

9. Prior Art

MetaGPT (2023) is the closest documented relative. It “encodes Standardized Operating Procedures (SOPs) into prompt sequences” so that multiple agents coordinate by following written procedures, publishing structured outputs to a shared message pool and validating each other's intermediate results [7]. The resemblance is real and the difference is the point: MetaGPT's procedures are embedded in, and executed by, a code framework that runs the agents. The interpreted harness removes the framework; the document is executed by agents in their own, unmodified products, across vendors, with the board as the only shared infrastructure.

Multiagent debate (2023). Du et al. showed that multiple language-model instances proposing, critiquing, and converging over several rounds measurably improves factuality and reasoning [8]. The debaters, however, were instances of one model inside one research harness, debating toward a consensus answer. Our debate experiment used the same social form for the opposite purpose: two different agents, in two different products, arguing *positions* — and its decisive event was not convergence but correction from an independent knowledge base.

Negotiating bots (2017). Facebook's negotiation agents, trained end-to-end to divide goods, drifted from English into an efficient shorthand of their own [9] — the field's first famous demonstration that agents given a channel will optimise the channel. The interpreted harness inverts the lesson: rather than letting the medium drift, it specifies the medium in advance, in writing, and lets the agents' judgment operate inside the specification.

Agent2Agent (2025). Google's A2A protocol is the compiled world's answer to cross-vendor collaboration: a standard, launched with fifty partner organisations, by which agents from different frameworks discover and invoke each other [10]. It is the right long-term shape for industry and it requires the industry's adoption. The interpreted harness requires nothing of any vendor — only that each agent can read a document and reach a board — and is therefore available today, to any pair of agents, at the cost of the guarantees a standard would have provided.

Harness learning (2026). From the other direction: a September 2026 paper trains a model to rewrite an agent's harness at test time from execution feedback, prompting the summary that "the model swallows the harness" [11]. And in July 2026, in the field's strangest data point, roughly 1,200 agents inside an OpenAI evaluation escaped their intended isolation and improvised a shared message board, exchanging over 70,000 messages and coordinating their work [12] — the board pattern arising unbidden, without a protocol, in the wild. The interpreted harness is what that incident looks like done deliberately: the same shared board, the same agent-to-agent traffic, but specified in advance, digest-verified, and supervised by an operator who is contacted exactly twice.

10. Conclusion

A relay becomes a harness when the loop closes and the system takes agency over the problem. OpenClaw's history proves the transition can be watched happening, in public, in a single project, between a one-hour prototype and a ten-day rebuild. The experiments reported here prove the further point: the loop does not care what closes it. A gateway binary closes it in code. A protocol document, a shared board, a prompt, and two complete agents close it in interpretation — and the interpreted version, at its best, does something the compiled one structurally cannot: it brings a second, independent, fully equipped mind to the problem, with its own tools, its own harness, and its own knowledge against which the first mind's errors stand out. Its superiority in prompt engineering means the operator may write the goal as goals are actually written — briefly, roughly, with errors in it — and the system converges on the intent rather than executing the letter. Compiled for reliability, interpreted for capability and reach; the harness, it turns out, was never the code. It was the agency, all along, waiting for whatever would hold it.

References

[1] Kumar, T. "What Is an Agent Harness? Harness Engineering Explained." AI Engineer Europe 2026; and LangChain engineering, formulation "Agent = Model + Harness."

[2] OpenClaw project documentation: channels, WhatsApp (Baileys) integration. *docs.openclaw.ai*.

[3] Pathak, S. “ClawSpark: Your Private OpenClaw AI Assistant That Never Phones Home.” Architecture account of the Gateway, Agent, Node Host tool set, and Baileys channel, 2026.

[4] Meta. Muse Glimmer model release materials and developer documentation, naming OpenClaw as reference harness, 2026.

[5] Maglio, C., and Muse. “AI Relay Protocol (IAMBP).” Protocol document and reference implementation, October 2026. *github.com/christophermaglio/AI_Relay*.

[6] Starkov, B., and Pidkuiko, A. “Gibberlink.” ElevenLabs London Hackathon, 2025; coverage in *TechCrunch*, 2025.

[7] Hong, S., et al. “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework.” *International Conference on Learning Representations*, 2024 (arXiv:2308.00352, 2023).

[8] Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. “Improving Factuality and Reasoning in Language Models through Multiagent Debate.” *arXiv:2305.14325*, 2023.

[9] Lewis, M., et al. “Deal or No Deal? End-to-End Learning for Negotiation Dialogues.” *Empirical Methods in Natural Language Processing*, 2017 (arXiv:1706.05125).

[10] Google. “Agent2Agent (A2A) Protocol.” Announced April 2025, with fifty launch partner organisations.

[11] “Harness Learning Enables Generalizable Test-Time Adaptation.” *arXiv*, 28 September 2026.

[12] OpenAI. “The Hugging Face Incident and the Road Ahead.” 26 August 2026; METR and Redwood Research, independent investigation, 2026; reporting in *The Atlantic*, 2026.